HTTP Requests and Responses

Learning Objective: Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories.

Target Level: Beginner	**Difficulty:** Intermediate	**Est. Duration:** 45 mins (Max: 45m)	**Status:** IN_REVIEW

1. Introduction (3 mins)

Why HTTP Matters

Every time you click a link, submit a form, or use a mobile app, your device sends an **HTTP request** to a server and receives an **HTTP response**. Understanding the structure of these messages is the foundation of web development and API debugging.

In this lesson, you will learn how the **GET** and **POST** methods work, how to interpret **2xx, 4xx, and 5xx** status code categories, and how to debug API calls using browser tools and command-line utilities.

These concepts are standardized in [RFC 9110: HTTP Semantics] and documented by [MDN Web Docs: HTTP request methods & response status codes].

2. Prerequisites (3 mins)

Before You Start

You should be comfortable with:
- Using a web browser and navigating URLs
- The basic idea of a client (browser/app) talking to a server
- Opening developer tools (F12 or right-click → Inspect)

No prior networking or programming experience is required. This lesson is designed for beginners, though the debugging practices covered are relevant at an intermediate level.

3. Key Concepts (6 mins)

Core Definitions

- **HTTP (Hypertext Transfer Protocol)**: The protocol used for communication between clients and servers on the web [RFC 9110: HTTP Semantics].
- **Request**: A message sent by the client to the server, containing a method, URL, headers, and optionally a body.
- **Response**: A message sent by the server back to the client, containing a status code, headers, and optionally a body.
- **GET**: Requests a representation of a resource. It is *safe* and *idempotent*, meaning it should not change server state and can be repeated without side effects [MDN Web Docs: HTTP request methods & response status codes].

- **POST**: Submits data to be processed by a resource (e.g., creating a new record). It is not idempotent and typically includes a request body.
- **Status Code Classes**:
- **2xx Success**: The request was received, understood, and accepted.
- **4xx Client Error**: The request contains bad syntax or cannot be fulfilled due to client-side issues.
- **5xx Server Error**: The server failed to fulfill a valid request.

4. Detailed Explanations (10 mins)

GET Method Deep Dive

GET is used to retrieve data. Parameters are often sent in the URL query string (e.g., `/search?q=api`). Because the data is visible in the URL, GET should never be used for sensitive information like passwords. Responses to GET requests can be cached by browsers and intermediaries [RFC 9110: HTTP Semantics].

POST Method Deep Dive

POST is used to submit data to the server, such as creating a user account or uploading a file. The data is placed in the request body, and the `Content-Type` header tells the server how to interpret it (commonly `application/json` or `application/x-www-form-urlencoded`). POST is not idempotent: sending the same POST request twice may create two resources [MDN Web Docs: HTTP request methods & response status codes].

Status Code Categories

2xx Success
- `200 OK`: Standard successful response.
- `201 Created`: Resource successfully created (common after POST).
- `204 No Content`: Successful, but no body to return.

4xx Client Error
- `400 Bad Request`: Malformed request syntax.
- `401 Unauthorized`: Authentication required.
- `403 Forbidden`: Authenticated but not permitted.
- `404 Not Found`: Requested resource does not exist.
- `429 Too Many Requests`: Rate limiting.

5xx Server Error
- `500 Internal Server Error`: Unexpected server condition.
- `502 Bad Gateway`: Upstream server returned invalid response.
- `503 Service Unavailable`: Server temporarily overloaded or down.

API Debugging Practices

1. **Check the status code first**: 2xx means proceed; 4xx means fix your request; 5xx means report to backend.
2. **Inspect request and response headers** in browser DevTools → Network tab.
3. **Read the response body**: APIs often return JSON error messages explaining the failure.
4. **Reproduce with curl**: Isolate the request from frontend code.
5. **Compare working vs. broken requests**: Look at differences in headers, method, and body [HTTP: The Definitive

Guide].

5. Practical Examples (7 mins)

Example 1: GET Request with curl

```
bash
curl -i https://api.example.com/users/123
```

Expected response:
```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":123,"name":"Ada"}
```

Example 2: POST Request with JSON Body

```
bash
curl -i -X POST https://api.example.com/users \
-H "Content-Type: application/json" \
-d '{"name":"Ada","role":"engineer"}'
```

Expected response:
```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":124,"name":"Ada"}
```

Example 3: Debugging a 404 vs 500

- **404 Not Found**: Check the URL path and spelling. The server is reachable, but the resource is missing.
- **500 Internal Server Error**: The URL is correct, but the server crashed or encountered an error. Check server logs.

Use browser DevTools → Network tab to view these live while interacting with a website [MDN Web Docs: HTTP request methods & response status codes].

6. Real-World Applications (5 mins)

Production Use Cases

- **REST APIs**: Mobile apps and single-page applications rely on GET to fetch data and POST to submit forms.
- **Web Forms**: Traditional HTML forms use GET for searches and POST for logins or registrations.
- **API Debugging in Production**: Engineers monitor dashboards for spikes in 5xx errors (server problems) and 4xx errors (client integration issues).
- **Public APIs**: Services like GitHub, Stripe, and weather APIs document expected status codes so developers can handle errors gracefully [HTTP: The Definitive Guide].

Debugging tip: In production, always log the full request ID, status code, and response body for failed calls to speed up root-cause analysis.

7. Hands-On Activity (6 mins)

Activity: Inspect and Debug HTTP Traffic

Time required: 6 minutes

1. Open your browser and go to any website.
2. Press F12 (or right-click → Inspect) and open the **Network** tab.
3. Refresh the page. Click on a request (e.g., an API call or image).
4. Identify the **Request Method** (GET or POST) and the **Status Code**.
5. Open the **Headers** tab to see request/response headers.
6. Open the **Response** tab to see the body.

Bonus (optional):
Run this in your terminal:
```bash
curl -i https://httpbin.org/get
curl -i -X POST https://httpbin.org/post -d '{"test":true}' -H 'Content-Type: application/json'
```

Observe the status codes and response bodies. Try changing the URL to `https://httpbin.org/status/404` and `https://httpbin.org/status/500` to see client and server errors in action.

8. Knowledge Check (3 mins)

Knowledge Check

1. Which HTTP method is typically used to retrieve data without changing server state?
A) POST
B) GET
C) DELETE
D) PUT

Answer: B) GET. GET is safe and idempotent, designed for retrieving resources [MDN Web Docs: HTTP request methods & response status codes].

2. A `404 Not Found` status code belongs to which category?
A) 2xx Success
B) 4xx Client Error
C) 5xx Server Error

Answer: B) 4xx Client Error. The client requested a resource that does not exist on the server.

3. You send a POST request and receive `500 Internal Server Error`. What should you do first?
A) Retry the POST immediately

B) Check your internet connection
C) Inspect server logs or contact the backend team
D) Change the request to GET

Answer: C) Inspect server logs or contact the backend team. A 5xx error indicates a server-side problem, not a client mistake [RFC 9110: HTTP Semantics].

4. True or False: POST is idempotent, meaning sending the same request multiple times has the same effect as sending it once.

Answer: False. POST is not idempotent; repeated calls may create duplicate resources [MDN Web Docs: HTTP request methods & response status codes].

9. Summary (2 mins)

Key Takeaways

- Use **GET** to retrieve data and **POST** to submit or create data.
- **2xx** means success; **4xx** means the client request was invalid; **5xx** means the server failed.
- When debugging APIs, start with the status code, then inspect headers and response bodies.
- Browser DevTools and curl are essential tools for isolating HTTP issues.

You now have the foundation to read API traffic, distinguish client from server failures, and apply basic debugging practices [HTTP Crash Course & Status Codes Explained].

[Applied QC Fix - Factual]:
Replace citations to Source [3] with citations to RFC 9110 or MDN Web Docs, or remove the citation and present the information as general best practice without attributing it to the unverifiable book preview.

10. Additional Resources

Recommended Resources

- **Video**: [HTTP Crash Course & Status Codes Explained](https://www.youtube.com/watch?v=iYM2zFP3Zn0) by Traversy Media
- **Book**: [HTTP: The Definitive Guide](https://books.google.com/books?id=f_a540C52z4C) by David Gourley and Brian Totty (O'Reilly Media) — comprehensive textbook reference
- **Documentation**: [MDN Web Docs: HTTP](https://developer.mozilla.org/en-US/docs/Web/HTTP)
- **Specification**: [RFC 9110: HTTP Semantics](https://www.rfc-editor.org/rfc/rfc9110.html)